PhotoMatch — Manual Técnico de Defensa

Trabajo Práctico 05: Testing Automatizado, Cobertura y Quality Gates

Ingeniería de Software III · Universidad Católica de Córdoba (UCC) · 2026

Alumno: Lautaro Alejo Vallino
Legajo: 2301009 · Comisión 2
Aplicación: PhotoMatch (Monolito Node.js/ PostgreSQL)

Release congelada: `v5.0.0`
Fecha límite entrega: 26 de octubre de 2026
Fecha defensa oral: 6 de noviembre de 2026 (13:30 hs)

Tabla de Contenidos

A. Introducción Conceptual . . . 2
1. La pirámide de testing y la distribución del esfuerzo . . . 2
2. Anatomía de un Unit Test: El patrón AAA (Arrange - Act - Assert) . . . 2
3. Dobles de prueba (Test Doubles): Stub vs Mock vs Fake . . . 2
4. Métricas de cobertura (Code Coverage) y sus límites . . . 2
5. Quality Gate (Puerta de Calidad) . . . 3
B. Estado Inicial de PhotoMatch antes del TP5 . . . 4
C. Implementación Paso a Paso . . . 4
D. Explicación Detallada del Código . . . 5
1. Inversión de Dependencias en `authService.js` . . . 5
2. El Test con Mock en `authService.test.js` . . . 5
E. Decisiones de Ingeniería y Alternativas . . . 6
El Ejercicio del Camino sin Cubrir (Requisito Obligatorio §1653-1660) . . . 6
F. Pruebas y Evidencias Reales del Repositorio . . . 7
1. Estado de la Suite de Pruebas Local . . . 7
2. Pull Request 1 (Demostración Rojo to Corrección to Verde to Merge) . . . 7
3. Pull Request 2 (El Gate Bloqueado Abierto para la Mesa Evaluadora) . . . 7
G. Preparación para la Defensa Oral . . . 8
Las 3 Preguntas Más Difíciles de la Mesa Evaluadora . . . 10
H. Resumen Final de Estudio . . . 11
Conceptos Fundamentales a Dominar . . . 11
Guión para la Exposición Oral (2 Minutos Reloj) . . . 11
Checklist de Demostración en Vivo . . . 11

A. Introducción Conceptual

El propósito del Trabajo Práctico 05 es transformar la calidad de la aplicación desde una noción abstracta o una intención artesanal hacia un **contrato medible y exigible por código** en el pipeline de Integración Continua (CI).

1. La pirámide de testing y la distribución del esfuerzo

En el desarrollo de software industrial, no todas las pruebas cuestan lo mismo ni entregan el mismo tipo de retroalimentación:

Punta: E2E (Playwright en TP7) — Muy pocas, lentas (segundos/minutos), frágiles, caras
Medio: Integración (API vs DB real en TP7) — Cantidad moderada, prueban colaboración
Base: Unit Tests (32 tests en TP5) — Muchas, corren en milisegundos (87 ms), aisladas, baratas

- **Unit Tests (Base):** Prueban unidades atómicas de lógica pura de forma aislada. Si un test unitario falla, el nombre del test indica inmediatamente qué regla de negocio y qué función se rompió.
- **Anti-patrón Ice Cream Cone (Pirámide Invertida):** Ocurre cuando un equipo prescinde de pruebas unitarias y construye únicamente pruebas E2E. El pipeline se vuelve extremadamente lento (horas), frágil ante cambios mínimos de UI (**flakiness**) y resulta imposible aislar la causa raíz del error sin depuración manual paso a paso.

2. Anatomía de un Unit Test: El patrón AAA (Arrange - Act - Assert)

Todo test unitario profesional se estructura en tres bloques claramente separados:

1. **Arrange (Preparar):** Inicializa los datos de entrada, los estados esperados y los dobles de riesgo (mocks o stubs).
2. **Act (Ejecutar):** Invoca la única función o método bajo prueba.
3. **Assert (Verificar):** Comprueba que los resultados devueltos y las interacciones con los dobles coincidan estrictamente con la especificación.

3. Dobles de prueba (Test Doubles): Stub vs Mock vs Fake

Para aislar una unidad de sus dependencias externas (bases de datos, red, APIs de terceros), se emplean dobles de riesgo:

- **Stub:** Objeto que suministra respuestas predefinidas fijas. Provee datos para que el código continúe, pero **no verifica nada**.
- **Mock:** Objeto que registra llamadas y sobre el cual el test **verifica la interacción mediante un assert** (ejemplo: «¿se llamó a `findByEmail` exactamente 1 vez y con el email en minúsculas?»).
- **Fake:** Implementación ligera pero funcional (por ejemplo, un repositorio en memoria con un `Map` en lugar de una base de datos PostgreSQL).

> **Diferencia Clave para la Mesa Evaluadora**
>
> La diferencia entre un Stub y un Mock no está en el objeto mismo, sino en el **Assert**: si el test comprueba cómo la función interactuó con la dependencia, es un **Mock**. Si sólo consume datos provistos por el doble, es un **Stub**.

4. Métricas de cobertura (Code Coverage) y sus límites

- **Line Coverage (Líneas):** Porcentaje de líneas de código fuente ejecutadas al menos una vez por los tests.
- **Branch Coverage (Ramas):** Porcentaje de bifurcaciones lógicas evaluadas (`if`, `else`, `||`, `&&`, operador ternario). Es la métrica más honesta y rigurosa.
- **La advertencia fundamental:** La cobertura mide **ejecución, no verificación**. Un test sin assertions puede recorrer el 100% de las líneas de una función y dar 100% de cobertura, pero no verifica nada. Cobertura alta con tests deficientes es falsa confianza.

5. Quality Gate (Puerta de Calidad) Un **Quality Gate** es una condición contractual verificada por el pipeline que frena automáticamente el merge de un Pull Request si no se alcanzan las métricas de calidad preacordadas (en nuestro caso, 80% de líneas y 70% de ramas).

B. Estado Inicial de PhotoMatch antes del TP5

Antes de implementar este trabajo práctico, el repositorio PhotoMatch presentaba las siguientes condiciones:

Componente	Estado Inicial	Problema o Carencia Técnica
Pipeline CI (`ci.yml`)	Solo compila imágenes Docker con Buildx (`push: false`).	No ejecutaba ningún test. Un PR con lógica rota o tests fallando se integraba en verde sin freno.
Autenticación (`authService`)	Importaba directamente `userRepository` con queries SQL fijas.	Imposible de probar sin una instancia activa de PostgreSQL en el puerto 5432.
Catálogo y Precios (`pricingService`)	Sin suite de pruebas unitarias.	Cálculo de packs (`single`, `pack6`, `all`) y regla de liquidación `isCounted` desprotegidos ante regresiones.
Herramientas de Cobertura	Inexistentes en `package.json`.	No se medían líneas ni ramas. No había reportes en Step Summary ni artefactos.
Arquitectura Frontend	Nginx que sirve CSS y actúa como reverse proxy hacia Express/EJS.	No existe SPA ni JavaScript cliente en el frontend (excepción formal de cátedra §1605).

C. Implementación Paso a Paso

1. **Inversión de dependencias en el backend:** Refactorizamos `backend/src/services/authService.js` permitiendo inyectar un repositorio alternativo como parámetro `userRepo = users`.
2. **Suite de pruebas con Mock (`authService.test.js`):** Escribimos 5 pruebas unitarias con patrón AAA utilizando un doble de riesgo para simular el repositorio de usuarios y verificar llamadas e inputs normalizados.
3. **Suite de pruebas parametrizadas y bordes (`pricingService.test.js`):** Creamos 16 pruebas unitarias con matrices de datos para validar cálculos de compras, packs y la regla `isCounted`.
4. **Instrumentación de cobertura con `c8`:** Añadimos `c8` a `backend/package.json` y creamos `.c8rc.json` con umbrales estrictos (80% líneas, 70% ramas) y exclusión de arranque e infraestructura.
5. **Extensión del pipeline en `.github/workflows/ci.yml`:** Incorporamos en `build-backend` la ejecución de `npm run test:check-coverage`, publicación del Step Summary y subida del artefacto `backend-coverage-report`.
6. **Demostración del Quality Gate (PR 16):**
   - **Commit 1 (`0f0f718`):** Subida de configuración de umbral sin los tests nuevos. El pipeline rompió por cobertura (55.81% líneas < 80%, 59.61% ramas < 70%). El PR quedó bloqueado.
   - **Commit 2 (`e4790fd`):** Incorporación de las suites de prueba. 32 tests pasaron, cobertura alcanzó 100% líneas y 70.58% ramas. El pipeline se puso en verde y se autorizó el merge a `main`.
7. **Demostración del Freno Permanente (PR 17):** Creación de la rama `demo/tp5-gate-bloqueado` con código nuevo sin tests. El PR permanece abierto y bloqueado en rojo hasta la defensa oral.

D. Explicación Detallada del Código

1. Inversión de Dependencias en `authService.js`

```
// backend/src/services/authService.js
const bcrypt = require('bcryptjs');
const users = require('../repositories/userRepository');

async function authenticate(email, password, userRepo = users) {
  const normalizedEmail = (email || '').trim().toLowerCase();
  if (!normalizedEmail) return null;

  const user = await userRepo.findByEmail(normalizedEmail);
  if (!user || !user.password_hash) return null;

  const passwordMatches = await bcrypt.compare(password || '', user.password_hash);
  if (!passwordMatches) return null;

  return { id: user.id, name: user.name, email: user.email, role: user.role };
}
module.exports = { authenticate };
```

Análisis técnico:

- `userRepo = users`: Provee un valor por defecto para entornos productivos (mantiene compatibilidad total con `authController.js`), pero permite inyectar un mock en los tests unitarios.
- `normalizedEmail`: Sanitiza el input eliminando espacios accidentales y forzando minúsculas. Previene errores tipográficos de login y consultas SQL innecesarias.
- `bcrypt.compare`: Emplea una comparación en tiempo constante que mitiga ataques de análisis de tiempos (**timing attacks**).
- Objeto sanitizado: Omite deliberadamente `password_hash` del objeto resultante para evitar fugas de credenciales en sesión o vistas.

2. El Test con Mock en `authService.test.js`

```
// backend/test/authService.test.js
test('authService - autenticación exitosa con mock de repositorio (AAA)', async () => {
  // Arrange: preparar el doble y datos simulados
  const plainPassword = 'passwordSegura123!';
  const passwordHash = bcrypt.hashSync(plainPassword, 4);
  const fakeUser = {
    id: 'usr-42', name: 'Fotógrafo Principal',
    email: 'fotografo@photomatch.com', role: 'admin', password_hash: passwordHash
  };
  const calls = [];
  const mockUserRepo = {
    findByEmail: async (email) => { calls.push(email); return fakeUser; }
  };

  // Act: ejecutar la función bajo prueba inyectando el mock
  const result = await authenticate('  FOTOGRAFO@photomatch.com  ', plainPassword, mockUserRepo);

  // Assert: verificar resultado e interacción con la dependencia
  assert.equal(calls.length, 1, 'findByEmail debió llamarse exactamente una vez');
  assert.equal(calls[0], 'fotografo@photomatch.com', 'Debe normalizar el email');
  assert.deepEqual(result, {
    id: 'usr-42', name: 'Fotógrafo Principal',
    email: 'fotografo@photomatch.com', role: 'admin'
  });
});
```

Por qué se escribió así: El objeto `mockUserRepo` captura en `calls` los argumentos de invocación. Los `assert` finales comprueban que el servicio interactuó correctamente con el repositorio antes de validar el resultado.

E. Decisiones de Ingeniería y Alternativas

Decisión	Alternativas Consideradas	Justificación Técnica y Consecuencias
Test Runner: `node:test`	Jest, Vitest, Mocha + Chai	Nativo de Node.js (cero dependencias externas). Arranca en 87 ms. No requiere transpilación ni plugins.
Mocking por Inversión de Dependencias	Monkey patching global (`proxyquire`, `sinon`)	Diseño desacoplado y explícito. No modifica el cargador de módulos de Node en runtime y hace el código intrínsecamente más limpio.
Instrumentación: `c8`	`nyc` (Istanbul tradicional)	Aprovecha la cobertura nativa del motor V8 de Node.js. No reescribe archivos en disco y falla inmediatamente el build mediante exit code 1.
Umbral: 80% Líneas / 70% Ramas	Fijar 100% o conformarse con 50%	Anclado en la medición real (100% líneas, 70.58% ramas). Evita la ley de Goodhart y garantiza que código nuevo sin tests frene el pipeline.
Exclusiones de Cobertura	Medir la carpeta raíz completa	Se excluye el arranque (`app.js`), infraestructura SQL (`db/`, `repositories/`) y vistas (`views/`). Mide lo que importa: la lógica pura.

El Ejercicio del Camino sin Cubrir (Requisito Obligatorio §1653-1660) Al analizar el reporte detallado generado por `c8`, identificamos la siguiente rama de decisión no ejercitada en el código:

1. **Archivo y línea exacta:** `backend/src/services/validationService.js`, líneas 30 y 31:

```
if (!COVERAGE_STATUSES.includes(data.status)) errors.push('El estado no es válido.');
if (!PAYMENT_STATUSES.includes(data.payment_status)) errors.push('El estado de pago no es válido.');
```

2. **Entrada concreta que recorrería esa rama:** Invocar `validateCoverage` con un objeto que contenga valores no contemplados en los arrays constantes, por ejemplo:

```
validateCoverage({ ...validCoverage, status: 'Inexistente', payment_status: 'NoExiste' });
```

3. **Decisión tomada y justificación:** Decidimos **no agregar este test en la suite unitaria**. En PhotoMatch, los estados válidos están fuertemente restringidos a nivel de dropdown select en las vistas EJS y a nivel de restricciones de dominio en PostgreSQL. Mantener esta rama sin cubrir deja la cobertura de ramas en un honesto **70.58%**, calibrando de manera óptima el umbral del Quality Gate (70%) para que cualquier código sin tests introducido en un PR rompa inmediatamente el build.

F. Pruebas y Evidencias Reales del Repositorio

> **Verificación en Vivo**
>
> Todas las evidencias presentadas a continuación son reales, comprobables y navegables en GitHub mediante sus identificadores únicos de commit, run y Pull Request.

1. Estado de la Suite de Pruebas Local

- **Comando:** `cd backend && npm run test:check-coverage`
- **Resultado: 32 tests unitarios pasando**, 0 fallos, duración total: 87 ms.
- **Métricas `c8`:**
  - Statements: 100% (86/86)
  - Branches: 70.58% (48/68)
  - Functions: 100% (9/9)
  - Lines: 100% (86/86)

2. Pull Request 1 (Demostración Rojo to Corrección to Verde to Merge)

- **Enlace al PR:** Pull Request #16 (feat: suite de pruebas y freno por cobertura)
- **Paso 1 — Corrida Roja por Umbral:**
  - **Commit:** `0f0f718`
  - **Enlace:** Actions Run #39985482676
  - **Comportamiento:** 11 tests pasaron, pero el step **Ejecutar tests unitarios y verificar umbral de cobertura (Quality Gate)** falló con exit code 1 al registrar 55.81% de líneas (umbral: 80%) y 59.61% de ramas (umbral: 70%). El required check `build-backend` quedó en rojo y bloqueó el PR.
- **Paso 2 — Corrida Verde tras Corrección:**
  - **Commit:** `e4790fd`
  - **Enlace:** Actions Run #39985690914
  - **Comportamiento:** Se incorporaron 21 tests en `authService.test.js` y `pricingService.test.js`. La cobertura subió al 100% en líneas y 70.58% en ramas. El build del backend y frontend pasaron en verde, se generó el Step Summary y se subió el artefacto `backend-coverage-report`.

3. Pull Request 2 (El Gate Bloqueado Abierto para la Mesa Evaluadora)

- **Rama:** `demo/tp5-gate-bloqueado`
- **Propósito:** Cumplir con la consigna de dejar un Pull Request abierto y frenado en rojo hasta el día de la defensa oral (6 de noviembre de 2026), permitiendo a los profesores verificar en vivo que la protección de rama (`build-backend` como Required Check) impide el merge cuando un cambio compila pero no alcanza la cobertura estipulada.

G. Preparación para la Defensa Oral

Pregunta 1: Dibujá la pirámide de testing. ¿Por qué tiene esa forma? ¿Qué sucede si se invierte?

Respuesta para defender:

Tiene base ancha y punta angosta porque el costo financiero, la velocidad de ejecución y la fragilidad varían radicalmente entre niveles. Los unit tests corren en milisegundos (nuestra suite corre 32 tests en 87 ms), son económicos y aíslan con precisión quirúrgica el fallo. Los tests E2E son lentos y propensos a falsos positivos. Si se invierte (**Ice Cream Cone**), el pipeline tarda horas, los desarrolladores ignoran los fallos intermitentes y es imposible diagnosticar qué línea rompió la app sin depurar manualmente.

Pregunta 2: Mostrame un test tuyo y señalá el Arrange, el Act y el Assert. ¿Por qué el nombre importa?

Respuesta para defender:

En `backend/test/authService.test.js` (test con mock): en el **Arrange** preparo el usuario simulado con hash bcrypt y el doble `mockUserRepo`; en el **Act** llamo a `authenticate` pasando el email con mayúsculas y espacios y el mock; en el **Assert** verifico que devuelva el usuario sin `password_hash`, que el mock se haya llamado 1 vez y que el email se haya normalizado a minúsculas. El nombre importa porque cuando el pipeline falla a las 3 AM en un servidor sin UI, el nombre del test es el primer diagnóstico.

Pregunta 3: ¿Qué diferencia hay entre un mock y un stub? Mostrame tu test con mock.

Respuesta para defender:

Un stub provee datos fijos para que la prueba corra, pero no verifica nada. Un mock registra las llamadas y sobre él realizamos asserts de interacción (cuántas veces fue llamado, con qué argumentos y en qué orden). En `authService.test.js`, mockeamos `findByEmail` de `userRepository` y verificamos que haya sido invocado exactamente una vez con `'fotografo@photomatch.com'`.

Pregunta 4: Line coverage vs Branch coverage: ¿cuál puede mentirte más y por qué?

Respuesta para defender:

Line coverage miente mucho más. Una sola línea puede contener una condición lógica múltiple como `if (esAdmin || tienePermiso)`. Ejecutar el test con `esAdmin = true` cubre la línea completa al 100%, pero la condición alternativa jamás fue evaluada. Branch coverage audita si cada rama (`true` y `false`) de todas las decisiones fue recorrida.

Pregunta 5: Tu cobertura es del 100% en líneas: ¿eso garantiza que tu código no tiene bugs?

Respuesta para defender:

No. Cobertura mide código **ejecutado**, no código **verificado**. Puedo escribir un test que llame a `calculatePlayerPurchase('pack6', 10)` sin ningún `assert`: la línea se marca como 100% cubierta, pero si la función devuelve un precio disparatado, el test pasa igual. Cobertura alta sin asserts es falsa confianza.

Pregunta 6: ¿Por qué elegiste 80% en líneas y 70% en ramas? ¿Qué pasa si mañana lo subís a 90%?

Respuesta para defender:

Elegí esos valores porque anclan la métrica en la realidad de nuestro proyecto (hoy medimos 100% líneas y 70.58% ramas en `src/services/`). Si mañana lo subiera a 90% en ramas, el build se rompería inmediatamente porque nos faltan cubrir ramas secundarias en `validationService.js` (estados inválidos), obligando a escribir tests de relleno solo para satisfacer el número (Ley de Goodhart).

Las 3 Preguntas Más Difíciles de la Mesa Evaluadora

🔴 **Pregunta Difícil 1:**
«Tenés 100% de line coverage en tus servicios. ¿Cómo podrías demostrar científicamente si tus tests son realmente rigurosos o si solo ejecutan código de forma superficial?»

Respuesta para defender:
Para verificar la efectividad real de los tests más allá de la métrica de cobertura se utiliza la técnica de **Mutation Testing (Pruebas de Mutación)** con herramientas como **Stryker**. El mutador introduce pequeñas alteraciones sintácticas deliberadas en el código de producción (**mutantes**), como invertir un operador booleano (`||` por `&&`), cambiar un borde (`>` por `>=`) o alterar un valor retornado. Luego corre la suite. Si algún test falla, el mutante es **eliminado (killed)**, demostrando que la suite es sensible a bugs reales. Si los tests siguen pasando, el mutante **sobrevive (survived)**, exponiendo que el test pasaba por esa línea sin verificarla con un assert adecuado.

🔴 **Pregunta Difícil 2:**
«En `isCounted(player, teamSale)`, si alguien cambia `!teamSale` por `teamSale`, ¿cuál de tus tests se rompe y por qué?»

Respuesta para defender:
Se rompe inmediatamente el test parametrizado de `pricingService.test.js` en el caso `cobertura sin teamSale: cada venta individual suma`. En ese caso de prueba, `teamSale` vale `false`, por lo que `!teamSale` evalúa a `true` y la función debe retornar `true`. Al invertir la condición a `teamSale`, la expresión evalúa a `false`, el assert `assert.equal(actual, expected)` recibe `false` en lugar de `true` y el pipeline falla de inmediato.

🔴 **Pregunta Difícil 3:**
«Tus guardianes de `main` son automáticos (build Docker y Quality Gate de cobertura). ¿Qué tipo de problemas NINGUNO de ellos puede detectar jamás? ¿Qué podría analizar una IA y qué queda exclusivamente para un humano?»

Respuesta para defender:
Ninguno de los guardianes automáticos puede detectar un **error de comprensión de requerimientos de negocio**. Si el cliente especificó que el Pack de 6 costara $15.000 pero el desarrollador interpretó $10.000 y codificó el producto y los tests con $10.000, tanto el build como los tests y la cobertura darán 100% en verde, pero la empresa cobrará mal a sus clientes.
Un agente de IA puede asistir comparando semánticamente el texto del requerimiento funcional contra el código para señalar discrepancias o sugerir casos de borde olvidados. Sin embargo, la **validación del valor de negocio, la aprobación de riesgos operacionales y el alineamiento con los objetivos estratégicos** sigue siendo una responsabilidad exclusivamente humana.

H. Resumen Final de Estudio

Conceptos Fundamentales a Dominar

1. **Pirámide de testing:** Base de unit tests baratos y rápidos; medio de integración; punta de E2E.
2. **Estructura AAA:** Arrange (preparación), Act (ejecución), Assert (verificación).
3. **Mocks vs Stubs:** El mock verifica interacciones y argumentos; el stub solo devuelve datos fijos.
4. **Line vs Branch coverage:** Las ramas auditan los caminos booleanos completos de cada decisión.
5. **Quality Gate:** Freno automático en CI que bloquea el merge si la cobertura cae por debajo del umbral.
6. **Inversión de dependencias:** Inyectar colaboradores externos para desacoplar el código y posibilitar pruebas unitarias aisladas.

Guión para la Exposición Oral (2 Minutos Reloj)

> **«En el TP5 transformamos la calidad de PhotoMatch en un contrato estricto verificado automáticamente en el pipeline de CI. > Partimos de una aplicación que únicamente tenía 11 pruebas de validación y un workflow de GitHub Actions que construía imágenes Docker pero no ejecutaba pruebas ni medía calidad. > Para resolverlo, refactorizamos el servicio de autenticación mediante inversión de dependencias, permitiéndonos sustituir la base de datos PostgreSQL por un mock para validar el aislamiento, la normalización de credenciales y la seguridad en el manejo de contraseñas. > Expandimos la suite a 32 pruebas unitarias que cubren 6 reglas de negocio críticas, incluyendo tests parametrizados para el catálogo de productos y liquidación de coberturas. > Incorporamos la herramienta `c8` y configuramos un Quality Gate en el job `build-backend` con umbrales de 80% en líneas y 70% en ramas sobre la lógica de negocio. Demostramos el funcionamiento del freno mediante dos Pull Requests: el PR 16, que evidenció el ciclo completo de freno por cobertura insuficiente (55.81%), corrección y merge exitoso al alcanzar 100% de líneas y 70.58% de ramas; y el PR 17, que permanece abierto y bloqueado en rojo para su verificación en vivo durante esta defensa.»**

Checklist de Demostración en Vivo

- [ ] **GitHub Repo:** Mostrar rama `main` y tag `v5.0.0`.
- [ ] **decisiones.md:** Abrir la sección del TP5 y mostrar la justificación de umbrales y el ejercicio del camino sin cubrir.
- [ ] **PR 16 (Mergeado):** Mostrar la conversación con la corrida roja `#39985482676` y la posterior corrida verde `#39985690914`.
- [ ] **Actions Run verde:** Mostrar el Step Summary con la tabla de cobertura formateada y el artefacto `backend-coverage-report` descargable.
- [ ] **PR 17 (Freno permanente):** Mostrar en vivo el botón de merge deshabilitado y el required check `build-backend` en rojo.
- [ ] **Código en VS Code:** Mostrar la inyección de `userRepo` en `authService.js` y el mock con verificación de llamadas en `authService.test.js`.
- [ ] **Terminal local:** Ejecutar `cd backend && npm run test:check-coverage` demostrando los 32 tests pasando en 87 ms con 100% de líneas y 70.58% de ramas.